

# Guía de cableado

Raspberry Pi · ESP32 · botones · actuadores

Sistema de Percheros Inteligentes

Instituto Tecnológico de Costa Rica

Área Académica de Ingeniería Mecatrónica

MT-7003 Microprocesadores y Microcontroladores

I Semestre 2026

# 1. Vista general del hardware

El sistema tiene tres bloques de hardware principales:

| Bloque                   | Componentes                                                   | Conectado a             |
|--------------------------|---------------------------------------------------------------|-------------------------|
| Entrada de usuario       | 4 botones momentáneos pulsador NA                             | GPIO de la Raspberry Pi |
| Procesamiento + interfaz | Raspberry Pi (modelo 3B+/4 recomendado), pantalla DSI o HDMI- |                         |
| Control de actuadores    | ESP32 DevKit, driver A4988/DRV8825, motor NEMA 17, 3 servos   | Raspberry Pi vía UART   |

La comunicación entre la Pi y el ESP32 es por UART (RS-232 lógico) a 115200 baud. La Pi lleva la base de datos y la GUI; el ESP32 controla los motores. Esta separación cumple el requisito del PDF de comunicación por RS-232/I2C/SPI exclusivamente.

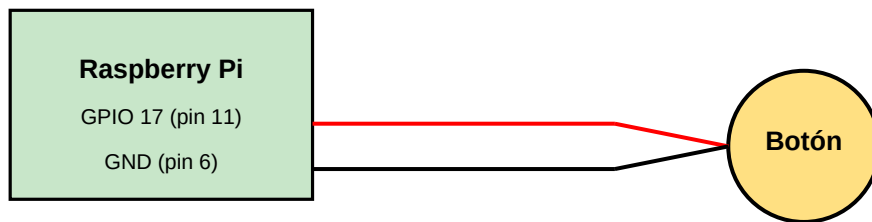
■ **Antes de cablear:** apaga la Pi y desconecta el ESP32 del USB. Verifica continuidad y polaridad con el multímetro antes de aplicar alimentación. Un cruce de TX-TX o de 5V con 3.3V puede dañar permanentemente cualquiera de los dos.

## 2. Cableado de los 4 botones

Los botones son pulsadores momentáneos normalmente abiertos. Cada botón tiene 2 cables: uno va a su pin GPIO de la Pi y el otro a GND (cualquiera de los pines GND de la Pi). El software activa el pull-up interno de cada GPIO, por eso cuando el botón está suelto el pin está en HIGH (3.3 V) y al presionarlo se conecta a GND y queda en LOW. La interrupción detecta este flanco descendente.

**NO se necesita resistencia externa** porque el pull-up interno del SoC ya cumple esa función.

### 2.1 Esquema individual de un botón



### 2.2 Pinout completo de los 4 botones

| Botón     | GPIO (BCM) | Pin físico de la Pi               | Función                              |
|-----------|------------|-----------------------------------|--------------------------------------|
| SI        | GPIO 17    | Pin 11                            | Confirmar selección / siguiente paso |
| NO        | GPIO 27    | Pin 13                            | Volver atrás / cancelar              |
| SIGUIENTE | GPIO 22    | Pin 15                            | Mover cursor adelante en una lista   |
| ANTERIOR  | GPIO 23    | Pin 16                            | Mover cursor atrás en una lista      |
| GND común | —          | Pin 6, 9, 14, 20, 25, 30, 34 o 39 | Cualquiera sirve                     |

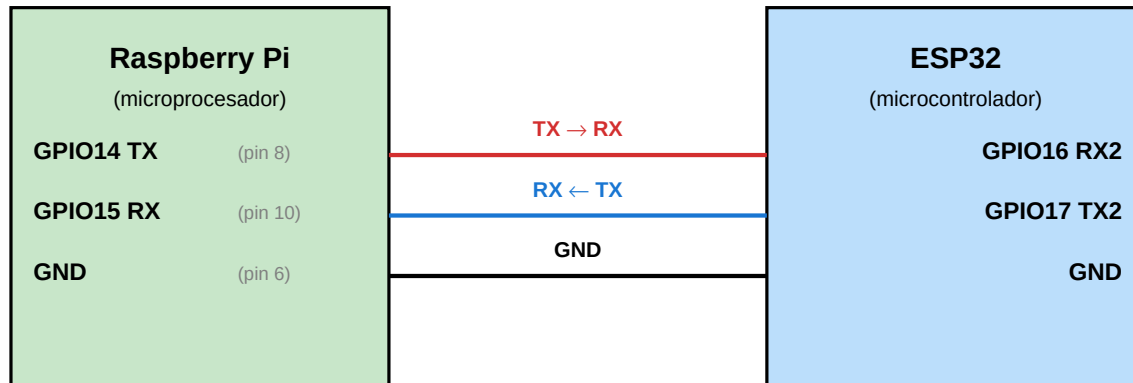
Estos pines se eligieron para no chocar con los buses I2C (GPIO 2/3), SPI (GPIO 7-11) ni UART (GPIO 14/15) que el ESP32 necesita más adelante. Si tienes un PCB con header 40 pines de Pi, los pines físicos 11/13/15/16 son contiguos y eso facilita armar un conector ribbon.

### 2.3 Anti-rebote por software

El software ya implementa anti-rebote por software con *bouncetime=200 ms* en `RPi.GPIO.add_event_detect`. Esto significa que después de detectar un flanco, ignora cualquier transición durante los 200 ms siguientes. Para botones de calidad con resorte interno esto es suficiente. Si experimentas doble-disparo, podrías agregar un capacitor de 100 nF en paralelo con el botón.

## 3. Comunicación UART Pi ↔ ESP32

### 3.1 Esquema de conexión



**Cable rojo:** TX de la Pi (GPIO14, pin 8) → RX2 del ESP32 (GPIO16).

**Cable azul:** RX de la Pi (GPIO15, pin 10) ← TX2 del ESP32 (GPIO17).

**Cable negro:** GND común entre Pi (pin 6) y ESP32 (cualquier GND).

■ **TX y RX se cruzan.** El TX de un dispositivo SIEMPRE va al RX del otro. Conectar TX-TX no daña nada pero la comunicación no funciona.

✗ **Sin conversor de niveles.** Pi y ESP32 son ambos lógicos 3.3 V, así que se conectan directo. Si usaras un Arduino Uno (5 V lógicos) sí necesitarías un conversor bidireccional o un divisor de tensión en la línea TX-Arduino → RX-Pi.

### 3.2 Tabla detallada

| Función   | Pi (BCM) | Pi (físico) | ESP32 (GPIO)            |
|-----------|----------|-------------|-------------------------|
| TX → RX   | GPIO 14  | Pin 8       | GPIO 16 (RX2)           |
| RX ← TX   | GPIO 15  | Pin 10      | GPIO 17 (TX2)           |
| GND común | GND      | Pin 6       | Cualquier GND del ESP32 |

### 3.3 Habilitar el UART en la Pi

Por defecto, en Raspberry Pi OS los pines UART están reservados para la consola serial. Hay que liberarlos:

```
sudo raspi-config

→ 3 Interface Options
  → I6 Serial Port
    → "Would you like a login shell to be accessible over serial?" No
    → "Would you like the serial port hardware to be enabled?" Yes

→ Finish, reboot
```

Después del reboot, /dev/serial0 apunta al UART hardware. Verifica con:

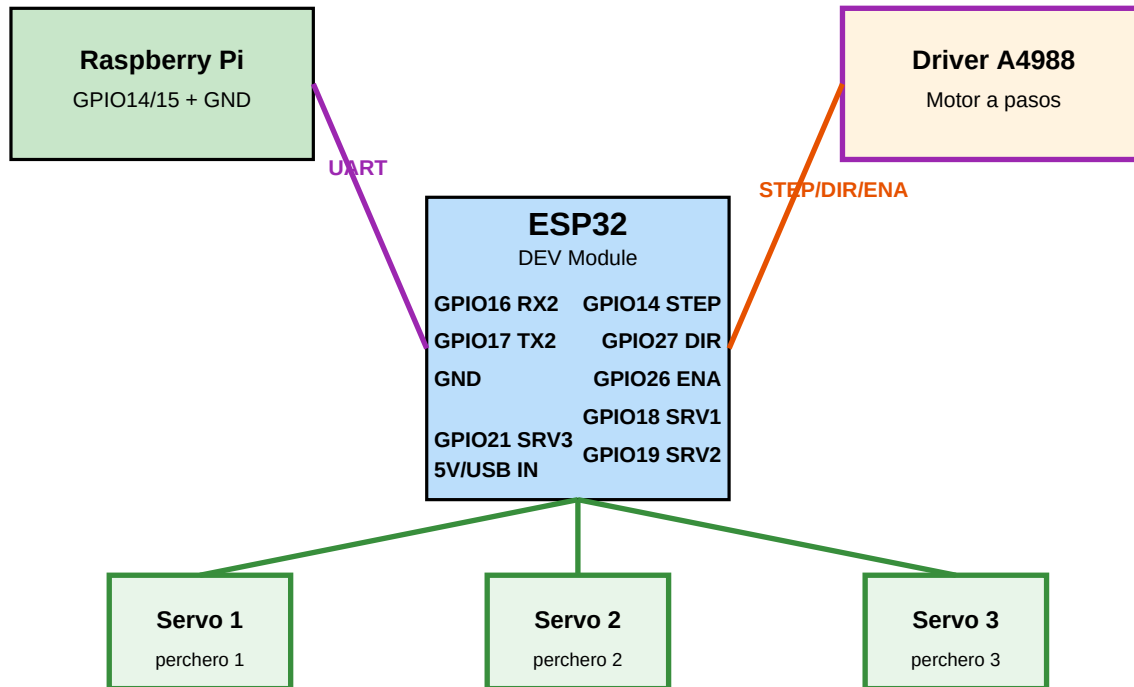
```
ls -l /dev/serial0
```

# debe ser un symlink → /dev/ttyAMA0 o /dev/ttyS0

### 3.4 Probar la conexión

```
# En la Pi:
sudo apt install python3-serial
python3 -c "
import serial
s = serial.Serial('/dev/serial0', 115200, timeout=2)
s.write(b'PING\n')
print(s.readline()) # debería imprimir b'OK\n'
"
```

## 4. ESP32 y actuadores



### 4.1 Pinout del ESP32

| Función          | GPIO ESP32 | Va hacia                                                |
|------------------|------------|---------------------------------------------------------|
| UART2 RX         | GPIO 16    | TX de la Pi (GPIO 14, pin 8)                            |
| UART2 TX         | GPIO 17    | RX de la Pi (GPIO 15, pin 10)                           |
| Stepper STEP     | GPIO 14    | STEP del driver A4988/DRV8825                           |
| Stepper DIR      | GPIO 27    | DIR del driver                                          |
| Stepper ENABLE   | GPIO 26    | ENABLE del driver (active LOW)                          |
| Servo perchero 1 | GPIO 18    | Cable de señal (naranja) del SG90 #1                    |
| Servo perchero 2 | GPIO 19    | Cable de señal (naranja) del SG90 #2                    |
| Servo perchero 3 | GPIO 21    | Cable de señal (naranja) del SG90 #3                    |
| GND común        | GND        | GND de Pi, GND de driver, GND de servos, GND de fuentes |

### 4.2 Driver A4988 (motor a pasos)

El driver A4988 (o DRV8825 equivalente) controla el motor a pasos NEMA 17. Conexiones obligatorias:

| Pin del A4988           | Va a                      | Notas                                                          |
|-------------------------|---------------------------|----------------------------------------------------------------|
| VMOT, GND (de potencia) | Fuente externa 8-35 V     | Pega capacitor electrolítico 100 $\mu$ F en paralelo (CRÍTICO) |
| VDD, GND (lógica)       | 3.3 V y GND del ESP32     |                                                                |
| STEP                    | GPIO 14 del ESP32         | Pulsos generados en interrupción                               |
| DIR                     | GPIO 27 del ESP32         | HIGH = una dirección, LOW = otra                               |
| ENABLE                  | GPIO 26 del ESP32         | LOW activa el driver                                           |
| MS1, MS2, MS3           | MS1=H, MS2=H, MS3=H       | Microstepping 1/8 (recomendado por suavidad)                   |
| 1A, 1B, 2A, 2B          | Bobinas A y B del NEMA 17 | Verifica color de cables del motor                             |
| RESET                   | Conectado a SLEEP         | (o ambos a 3.3 V para mantener activo)                         |

■ **Capacitor de 100  $\mu$ F entre VMOT y GND es OBLIGATORIO.** Sin él, los picos de corriente del motor pueden destruir el chip A4988 al primer arranque. Polaridad: el negativo (banda blanca) va al GND.

Antes de cablear el motor, ajusta la corriente del driver girando el potenciómetro pequeño del A4988. Para un NEMA 17 de 1.7 A típico,  $V_{ref} \approx 0.85$  V medido entre el potenciómetro y GND. Esto se hace con el motor desconectado y la lógica alimentada.

### 4.3 Servomotores (SG90 o MG90S)

| Cable del servo | Color típico       | Va a                         |
|-----------------|--------------------|------------------------------|
| Señal (PWM)     | Naranja / Amarillo | GPIO del ESP32 (18, 19 o 21) |
| Alimentación +  | Rojo               | 5 V de fuente externa        |
| Alimentación –  | Café / Negro       | GND común con ESP32 y fuente |

**NO alimentes los servos por el pin 5V/3.3V del ESP32.** Cada SG90 puede pedir picos de 600 mA al arrancar; los tres juntos podrían llegar a 1.8 A y eso resetea al ESP32. Usa una fuente externa de 5 V con al menos 2 A, y conecta su GND con el GND del ESP32.

# 5. Esquema de alimentación

El sistema tiene tres dominios de alimentación independientes pero con GND común. Esto evita que el ruido del motor a pasos se propague a la lógica de la Pi y del ESP32.

| Dominio          | Voltaje                           | Fuente sugerida                         | Qué alimenta                        |
|------------------|-----------------------------------|-----------------------------------------|-------------------------------------|
| Lógica Pi        | 5 V / 3 A                         | Fuente oficial Pi (USB-C / micro-USB)   | Basquero y Pi                       |
| Lógica ESP32     | 5 V (USB) o 3.3 V (regulador USB) | Cable USB de la PC o fuente 5 V por USB | ESP32 y la lógica del A4988 (3.3 V) |
| Potencia motores | 12 V / 3 A                        | Fuente switching DC 12 V                | VMOT del A4988 (motor a pasos)      |
| Potencia servos  | 5 V / 2 A                         | Fuente switching DC 5 V dedicada        | Pin rojo de los 3 servos            |

**Todos los GND deben estar unidos.** Conecta los GND de las cuatro fuentes (Pi, ESP32, motor, servos) a un punto común — típicamente una regleta o un punto en una protoboard. Si los GND no comparten referencia, las señales lógicas no significan nada y nada va a funcionar.



## 6. Lista de componentes (Bill of Materials)

| Cant. | Componente              | Especificación mínima           | Nota                                     |
|-------|-------------------------|---------------------------------|------------------------------------------|
| 1     | Raspberry Pi            | Modelo 3B+, 4 o 5               | Con tarjeta SD $\geq$ 16 GB              |
| 1     | ESP32 DevKit            | ESP32-WROOM-32                  | Cualquier placa de desarrollo con USB    |
| 4     | Pulsador momentáneo NA  | Tactile switch 6×6 mm o similar | Para los 4 botones                       |
| 1     | Driver paso-paso        | A4988 o DRV8825                 | Con disipador, sino se sobrecalienta     |
| 1     | Motor a pasos           | NEMA 17, 1.7 A, 1.8°/paso       | (p.e. 17HS19-1684S)                      |
| 3     | Servomotor              | SG90 (9 g, 1.8 kg·cm)           | Para una versión más robusta usar MG90S  |
| 1     | Capacitor electrolítico | 100 $\mu$ F / 35 V              | Crítico entre VMOT y GND del A4988       |
| 1     | Fuente switching        | 12 V DC / 3 A                   | Para alimentar VMOT del A4988            |
| 1     | Fuente switching        | 5 V DC / 2 A                    | Para los 3 servos                        |
| 1     | Cable Dupont set        | M-M, M-H, H-H                   | Mínimo 40 cables de cada tipo            |
| 1     | Protoboard              | 830 puntos                      | Para prototipar conexiones               |
| 1     | Multímetro              | Cualquiera con continuidad      | Para verificar antes de aplicar potencia |

# 7. Procedimiento de armado

Sigue este orden estricto. Cada paso solo aplica energía después de verificar el anterior con multímetro.

| Paso | Acción                                                           | Verificación                                                       |
|------|------------------------------------------------------------------|--------------------------------------------------------------------|
| 1    | Cablea los 4 botones a sus GPIO en la Pi (todo apagado)          | Continuidad entre cada GPIO y GND solo cuando se presiona el botón |
| 2    | Cablea UART Pi↔ESP32 (TX↔RX cruzados, GND común)                 | Continuidad TX-Pi a RX-ESP32 y GND común                           |
| 3    | Alimenta solo Pi y ESP32 (USB), corre `python3 main.py`          | GUI aparece, los 4 botones navegan correctamente                   |
| 4    | Sube el firmware al ESP32 con Arduino IDE                        | Monitor serial muestra '[ESP32] Firmware iniciado'                 |
| 5    | Quita el flag --mock, corre `python3 main.py`                    | PING ESP32: OK aparece en consola                                  |
| 6    | Cablea el A4988 lógico (sin VMOT, sin motor)                     | ESP32 sigue respondiendo a PING                                    |
| 7    | Calibra Vref del A4988 (~0.85 V para NEMA 17 1.7A)               | Vref estable medido entre el potenciómetro y GND                   |
| 8    | Conecta el motor al A4988 (con motor en seco, sin carga)         |                                                                    |
| 9    | Aplica VMOT 12V con capacitor 100 µF                             | Driver no se calienta excesivamente al reposo                      |
| 10   | Manda MOVE_BASE 2 desde la GUI (Información→Vejicular y luego 2) | Velocidad y tiempo adecuadamente                                   |
| 11   | Cablea los 3 servos con su fuente 5V dedicada                    | Continuidad GND con el ESP32                                       |
| 12   | Manda PRESENT 1 0..4 desde la GUI                                | Servo del perchero 1 visita las 5 posiciones                       |
| 13   | Repite con percheros 2 y 3                                       | Los tres servos responden a sus comandos                           |

## 8. Solución de problemas

| Síntoma                                  | Causa probable                                             | Solución                                                                        |
|------------------------------------------|------------------------------------------------------------|---------------------------------------------------------------------------------|
| GUI no responde a botones                | Pull-up no se está activando, o botón cableado mal         | Verifica con un multímetro que el GPIO esté en 3.3 V suelto y activa el pull-up |
| PING ESP32 timeout                       | TX/RX cruzados al revés o UART no ha sido inicializado     | Revisa el pin config; intercambia los cables TX/RX                              |
| ESP32 se resetea al mover servos         | Servos están alimentados por el ESP32                      | Usa fuente externa 5V dedicada para los servos                                  |
| Motor a pasos vibra pero no gira         | Bobinas mal conectadas (orden 1A,1B,2A,2B)                 | Revisa la resistencia entre cables del motor para identificar las bobinas       |
| A4988 se sobrecalienta y huele a quemado | Falta capacitor 100 $\mu$ F, o Vref demasiado alta         | Añade un 100 $\mu$ F y revisa antes de seguir                                   |
| Servo vibra y no llega a la posición     | Alimentación insuficiente o cable de señal demasiado largo | Conecta a una fuente 5V/2A, acorta cable de señal                               |
| Comando ROTATE 1 5 retorna ERR           | Rot 5 no existe (rango válido 0..4)                        | Es comportamiento correcto; el cliente Python ya lo valida                      |
| estado.txt corrupto                      | Edición manual incorrecta                                  | El sistema valida al cargar; si falla, mantiene el estado anterior              |

Para depurar la comunicación con el ESP32 puedes abrir el Monitor Serial de Arduino IDE (puerto USB del ESP32, 115200 baud) mientras la Pi le manda comandos: el firmware imprime cada comando recibido y cada respuesta enviada con flechas ← y →.